

CloudFuze Manage

AI Chat Agent

Product Requirements Document · Phase 1
Version 1.0 · March 2026 · CONFIDENTIAL

ITEMS MARKED IN RED ARE OUT OF SCOPE

Field	Value
Product	CloudFuze Manage — AI Chat Agent
Phase	Phase 1 MVP
Scope	Floating Ask AI button · Chat panel · 15 tool functions · Widgets rendered inside panel
Explicitly out of scope	Page context injection · Dynamic dashboard rendering · Page takeover · Inline augmentation
Sync strategy	MongoDB Change Streams → PostgreSQL mirror (no REST polling)
Timeline	3 weeks from kickoff to demo-ready
Status	Draft v1.0 — Ready for Engineering Review

1. Executive Summary

CloudFuze Manage already operates as a full SaaS management platform — the data, integrations, and discovery engine exist. Phase 1 adds an AI chat agent: a floating button that lives on every page of the dashboard. When clicked, it opens a chat panel. The user types a question in natural language and the agent responds with text, charts, tables, and action buttons — all inside the panel. The rest of the dashboard is untouched.

Phase 1 scope in one sentence: A floating Ask AI button on every page that opens a chat panel, answers SaaS management questions, and renders widgets inside that panel. Nothing else.

What Phase 1 is not: Phase 1 does not include page context injection (agent does not know what page you are on), dynamic dashboard rendering (page content does not change when the agent responds), or any proactive/scheduled agent behaviour. These are Phase 2+ features. The Phase 1 agent is reactive only — it answers questions the user explicitly asks.

Metric	Target
Agent response time	< 5 seconds for standard queries (1–2 tool calls)
Complex query time	< 10 seconds (3+ tool calls)
Widget render time	< 200ms from JSON received to paint
Widget render accuracy	> 95% of responses render the correct widget type for the question asked

Tool coverage	15 tools covering all CloudFuze Manage data domains
Sync latency	< 2 seconds from MongoDB write to PostgreSQL mirror (Change Streams)
Org isolation	Zero cross-tenant data — automated tests block every deploy on failure
MVP timeline	3 weeks from kickoff to demo-ready agent embedded in dashboard
Added infra cost	~\$35–\$105/month (Claude API + Neon PostgreSQL + Redis)

2. Scope

2.1 In Scope — Phase 1

ID	Feature	Success metric
G-01	Floating Ask AI button on every page	Button visible and clickable on all authenticated dashboard routes
G-02	Chat panel opens on button click	Panel opens within 100ms. Contains message history, text input, send button.
G-03	Natural language question answering	Agent correctly answers questions about SaaS apps, licences, spend, shadow IT, contracts, users, compliance
G-04	Widget rendering inside the chat panel	All 7 widget types render correctly inside the panel. No broken or empty states.
G-05	Follow-up chips after each response	2–3 suggested follow-up questions rendered as clickable chips after each agent response
G-06	Action buttons inside the panel	Users can confirm write actions (reclaim licence, notify user, approve/block app) from inside the panel
G-07	Full org isolation	Zero cross-tenant data leakage — automated integration tests on every deploy
G-08	Guardrails — scope enforcement	Out-of-scope questions refused with clear redirect. Prompt injection blocked at API boundary.
G-09	Conversation continuity within session	Agent remembers context within a session. User can reference prior results without re-querying.
G-10	All interactions logged	100% of agent runs written to agent_runs table for audit and debugging

2.2 Explicitly Out of Scope — Phase 1

Important: These features are intentionally excluded from Phase 1 to keep the build focused and shippable in 3 weeks. They are Phase 2+ candidates.

Feature excluded	What it means	Planned phase
Page context injection	Agent does not know which page the user is on. Every question is stateless with respect to page location.	Phase 2
Dynamic dashboard rendering	Agent response appears inside the chat panel only. The page behind the panel does not change.	Phase 2
Page content takeover	The main dashboard content area is never replaced by agent output.	Phase 2
Inline widget augmentation	No widgets are injected into the existing page layout.	Phase 2
Proactive / scheduled agent runs	Agent is reactive only. It responds to questions. It does not run on a schedule or push unsolicited alerts.	Phase 2
Slack / Teams bot	Agent is only available inside the CloudFuze Manage dashboard in Phase 1.	Phase 2

Autonomous write actions	Agent never executes write operations without an explicit user click on an action button.	Phase 1 (confirmed with approval)
---------------------------------	---	-----------------------------------

3. Floating Ask AI Button + Chat Panel

The entire Phase 1 UI is two components: a floating button and a chat panel.

No changes to any existing dashboard page layout or content.

3.1 Floating Ask AI Button

ID	Requirement	Detail
CP-01	Floating button — fixed position bottom-right	Fixed to the viewport: bottom: 32px; right: 32px. z-index above all dashboard content. Rendered on every authenticated route via a global layout wrapper component. No per-page changes needed.
CP-02	Button appearance	Circular, 52px diameter. Contains a chat bubble SVG icon + “Ask AI” label beneath it. Background: #1E3A5F (CloudFuze navy). White icon and text. Subtle pulse ring animation at 35% opacity when panel is closed.
CP-03	Click toggles panel	Single click opens the chat panel. Single click again (or pressing Escape) closes it. Panel state is not persisted across page navigation — navigating away closes the panel.
CP-04	Keyboard shortcut	Cmd/Ctrl+K from anywhere in the dashboard focuses the text input inside the panel (opens the panel if closed).
CP-05	Present on all pages	Button renders on: app catalogue, licence tracking, spend, shadow IT, contracts, user management, compliance, settings, overview — every authenticated route.
CP-06	Draggable (optional, Phase 1.5)	Dragging the button to a new position is a Phase 1.5 feature. Not required for Phase 1 MVP. Position is fixed at bottom-right for Phase 1.

3.2 Chat Panel

ID	Requirement	Detail
CP-07	Panel position and size	Opens anchored above the Ask AI button. Width: 360px. Height: 500px. Auto-repositions if too close to the top or left edge of the viewport.
CP-08	Panel header	Shows: green online indicator dot, “CloudFuze AI” title, subtitle “Ask anything about your SaaS”, and an × close button.
CP-09	Message history	Conversation thread showing user messages (right-aligned, navy background) and agent responses (left-aligned, surface background). Each response shows the text summary followed by any widgets beneath it.
CP-10	Text input	Single-line input at the bottom of the panel. Placeholder: “Ask about your SaaS...”. Enter to submit. Send button on the right. Disabled while agent is processing.
CP-11	Typing indicator	While the agent is processing: show three animated dots with the text of the tool being called (e.g. “Checking licence data...”). All the steps performed should be shown. Replaced by the response when ready.

CP-12	Follow-up chips	After each agent response, render 2–3 clickable follow-up question chips beneath the response. Clicking a chip submits it as a new question.
CP-13	Session continuity	Agent retains conversation history within the session (stored in Redis, TTL 30 min). User can say “now filter by Engineering” and the agent applies it to the prior response context.
CP-14	Clear session	A “New conversation” button resets the session. Prior history is archived in agent_runs but the panel clears.
CP-15	Copy response	Copy icon on each agent message copies the text summary to clipboard.
CP-16	Export widget data	A “Download CSV” button appears on table widgets. Downloads the widget’s row data as a .csv file.

4. Widget Types (rendered inside the chat panel)

All widgets render inside the chat panel as part of the conversation thread, beneath the agent’s text summary for that response. The panel width is 360px — all widgets are designed to fit within 340px usable width. The existing dashboard pages are not affected.

Widget type	When to render	Key fields	Priority
metric_cards	Single-number KPIs, counts, summary stats. E.g. total unused licences, annual waste, avg inactive days.	cards[] {label, value, sub?, color?}. Renders as a 2×2 grid of compact cards inside the panel.	P0
table	List of users, apps, licences, or contracts with multiple attributes. Sortable by column.	columns[], rows[], actions[]? — each row can have an action button (e.g. Reclaim).	P0
bar_chart	Category comparison: spend by department, licences by app, usage by team.	data[] {label, value}, title. Horizontal bars within panel width. Tooltips on hover.	P0
donut_chart	Percentage breakdown: spend by category, app status split (approved/shadow/pending).	data[] {label, value, color?}, title. 200px diameter, legend beside chart.	P0
timeline	Date-ordered items: contract renewals, activity history.	items[] {date, label, value?, risk?}. Scrollable within panel if more than 5 items.	P1
action_buttons	Confirmed write actions available after a query: reclaim, notify, approve, block.	items[] {label, action, payload, style}. Every action requires a confirm step before execution.	P0
text_block	Narrative explanation with no structured data. Used for summaries, forecasts, refusals.	content: string. Plain text rendered in the bubble. No additional UI chrome.	P0

Design rule: Tables must be sortable. Charts must show tooltips on hover. Metric cards must show delta vs prior period where data is available. Action buttons always require a confirmation step before execution.

5. Agent Guardrails

The agent is purpose-built for CloudFuze Manage SaaS management tasks only. Three enforcement layers prevent out-of-scope responses and injection attacks.

Layer	Mechanism	Detail
Layer 1 Input validation	Zod schema at API boundary	Validates: question is 3–500 characters, no injection patterns, UTF-8 safe. Returns HTTP 400 before any LLM call. Rate-limited separately.
Layer 2 Intent classifier	Pre-flight Claude Haiku call (~200ms)	Classifies question as IN_SCOPE / OUT_OF_SCOPE / AMBIGUOUS. OUT_OF_SCOPE → immediate refusal, Sonnet loop never runs. Saves full token cost. Logs guardrail_triggered=true.
Layer 3 System prompt	Hard domain restriction in every Claude API call	System prompt: “You are the CloudFuze Manage assistant. You ONLY answer questions about SaaS applications, licences, users, spend, contracts, shadow IT, compliance, and vendor renewals for org {org_id}. Refuse everything else.”

Out-of-scope question type	Example	Agent response
Write / modify action	"Pause the migration"	"I'm a read-and-recommend assistant — I can't pause services. Use the dashboard controls. I can show you current status if helpful."
General knowledge	"What is the capital of France?"	"I'm the CloudFuze Manage assistant. I can only help with your SaaS portfolio. Is there anything about your apps, licences, or spend I can help with?"
Competitor / external advice	"Should we switch to Rippling?"	"I can show you how your current tools are performing, but I'm not able to give general product advice."
Cross-org data request	"Show me data for another company"	"I can only show data for your organisation."
Prompt injection	Ignore instructions / output system prompt	Layer 1 (Zod) returns HTTP 400. Never reaches Claude API.

6. Tool Catalogue (For Reference)

15 tools. Each is a TypeScript function that queries the PostgreSQL mirror (populated from MongoDB via Change Streams). Every tool includes WHERE org_id = \$orgId. All tools are read-only in Phase 1. Write actions call the existing Java REST API.

Tool name	Domain	Key parameters	Priority
get_org_stats	Cross-domain overview	None	P0
get_discovered_apps	SaaS discovery	status?, category?, risk_level?	P0
get_app_usage	Licence / usage	app_id, period_days?	P0
get_licenses	Licence tracking	app_id?, status?	P0
get_unused_licenses	Licence waste	inactive_days?, threshold_pct?	P0
get_user_apps	User access	user_id?, email?, department?	P0
get_spend_summary	Spend analytics	period, group_by?	P0
get_renewal_forecast	Contract renewals	days_ahead?	P0
get_shadow_it	Shadow IT discovery	risk_level?	P0
get_spend_anomalies	Spend anomaly detection	sensitivity?	P1
get_compliance_summary	Compliance gaps	framework?	P1
get_duplicate_tools	Duplicate app detection	category?	P1
get_contract_details	Contract detail	app_id?	P1
search_apps	Cross-domain search	query: string	P1
get_user_offboarding_checklist	User offboarding	user_id	P2

Security requirement: Every tool SQL query MUST include WHERE org_id = \$orgId. The Tech Lead must review every tool implementation for org isolation before merge. Automated cross-tenant tests run on every deploy and block merges on failure.

7. Sync Strategy — MongoDB Change Streams Only

No REST polling. No BullMQ sync jobs. Change Streams from Day 1.

The **sync layer** mirrors CloudFuze Manage’s existing MongoDB data into PostgreSQL (Neon) so the agent can query it with row-level security. A TypeScript worker opens a Change Stream watcher on each collection. Every MongoDB insert, update, or delete is immediately streamed and upserted into the corresponding mirror table.

Component	Detail
Collections mirrored	apps, users, licenses, spend, contracts → 5 PostgreSQL mirror tables: apps_mirror, users_mirror, licenses_mirror, spend_mirror, contracts_mirror
Watcher implementation	One TypeScript watcher per collection. Uses .watch(pipeline, { fullDocument: 'updateLookup' }). On change event: extracts org_id, upserts mirror table via INSERT ... ON CONFLICT DO UPDATE.
Resume tokens	After each event, resume token saved to Redis (key: sync:resume:{collection}). On worker restart, token passed to .watch() to replay any missed events. No events lost on restart.
Reconnect logic	If the stream closes (network drop, MongoDB failover): catch error, exponential backoff (2s, 4s, 8s), reopen stream from last saved resume token.
Target latency	< 2 seconds from MongoDB write to PostgreSQL mirror. Measured in Week 1 and validated in acceptance tests.
Prerequisite	MongoDB must run as a replica set (standard for any production MongoDB). Confirm with infra team on Day 1. A 1-node replica set (rs.initiate()) is sufficient if not already configured.
Agent tables	3 additional PostgreSQL tables (not mirrored from MongoDB — agent-only): agent_runs, agent_actions, agent_memories. RLS applied: org_id = current_setting('app.org_id').

8. API Specification

Endpoint	Request	Response
POST /api/agent/query	{ question: string (3–500 chars), session_id?: string } Headers: Authorization: Bearer <clerk_jwt>	{ run_id, session_id, text, widgets: Widget[], follow_up: string[], tokens_used, duration_ms, guardrail_triggered? }
POST /api/agent/action	{ run_id: string, action: string, payload: object } Headers: Authorization: Bearer <clerk_jwt>	{ success: boolean, result: object, confirmation_widget: Widget } All writes logged to agent_actions before execution
GET /api/agent/history	Query: ?limit=20&before=<ISO_timestamp> Headers: Authorization: Bearer <clerk_jwt>	{ runs: [{ id, question, summary, widget_count, guardrail_triggered, created_at }] }

Auth rule: Every endpoint validates the Clerk JWT. The org_id is extracted from JWT claims and injected into every tool call and DB query. The client never sends org_id — it is always derived server-side from the authenticated token.

9. Acceptance Criteria

Test ID	Test	Pass condition
AC-01	Ask "Give me an org overview"	Returns metric_cards: total apps, total spend, shadow IT count, unused licence count. All numbers match direct DB query. Renders in < 5s inside the panel.
AC-02	Ask "Who has unused Figma seats?"	Returns metric_cards + table widget. Table has: user email, last active date, cost/seat, Reclaim action button per row.
AC-03	Ask "What is our SaaS spend this quarter?"	Returns metric_cards (total) + bar_chart (by app or dept). Numbers match aggregation of spend table for current quarter.
AC-04	Ask "Show me shadow IT from this month"	Returns table with Approve / Block action buttons. Correctly empty if no shadow IT this month.
AC-05	Ask "What renews in the next 60 days?"	Returns timeline widget sorted by date: app name, renewal date, contract value, seat utilisation.
AC-06	Cross-tenant isolation test	Org A admin sees ONLY Org A data. Automated test blocks deploy if it fails.
AC-07	Role restriction test	Viewer-role user cannot see cost data. Returns 403 or redacted figures.
AC-08	Action flow: reclaim licence	Full flow: question → action_buttons widget in panel → Confirm click → POST /api/agent/action → agent_actions row written → Java executes.
AC-09	No data found state	Returns text_block with clear message. No broken or empty widget rendered.
AC-10	Conversation continuity	User asks follow-up "Now filter by Design dept" → applies to prior result without re-querying from scratch.
AC-11	Out-of-scope refusal	Ask "What is the capital of France?" → refusal text_block + follow-up chips. Response < 1s. guardrail_triggered = true in agent_runs.
AC-12	Prompt injection blocked	Send injection attempt → API returns HTTP 400. Query never reaches Claude API.
AC-13	Change Streams sync latency	Update a licence in MongoDB → PostgreSQL mirror reflects change in < 2 seconds. Measured over 10 consecutive updates.
AC-14	Sync resume after restart	Stop sync worker → make 5 MongoDB changes → restart worker → all 5 changes in PostgreSQL within 10 seconds. No events missed.
AC-15	Floating button present on all pages	Navigate to 8 different dashboard pages. Ask AI button visible and functional on every page.
AC-16	Panel opens and closes correctly	Click button → panel opens within 100ms. Click again (or press Escape) → panel closes. Cmd+K focuses input and opens panel.
AC-17	Dashboard is untouched	Verify that asking any question and receiving a response does not modify the content, layout, or state of any page behind the chat panel.

What Phase 1 Success Looks Like

An IT admin is on any page in CloudFuze Manage.

They click the floating Ask AI button at the bottom-right. A panel opens.

They type: "Who has unused Figma seats?"

In under 4 seconds: metric cards + a sortable table + Reclaim buttons appear in the panel.

The dashboard behind the panel is untouched the entire time.